Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №3
По дисциплине: «КиСБ»
Тема: «**Блочные шифры и классические режимы шифрования**»

Выполнил:
Студент 4-го курса
Группы АС-66
Горобец А.В.
Проверил:
Хацкевич А.С.

Брест 2026

Вариант 3

Цель работы: программная реализация симметричного блочного шифра (AES-128 или СТБ 34.101.31 «БелТ») и классических режимов работы (ECB, CBC, CFB, OFB, CTR) с поддержкой алгоритма дополнения блоков (Padding).

Задание:

Базовый шифр: AES-128

Режимы работы: ECB + CFB

Особенности реализации / Фокус тестирования: Демонстрация использования исключительно функции E_K для расшифрования в режиме CFB.

Код программы:

```
package lab3;

public class AES128 {

    private static final int BLOCK_SIZE = 16;
    private static final int KEY_SIZE = 16;
    private static final int ROUNDS = 10;

    private static final int[] SBOX = {
            0x63, 0x7c, 0x77, 0x7b, 0xf2, 0x6b, 0x6f, 0xc5,
            0x30, 0x01, 0x67, 0x2b, 0xfe, 0xd7, 0xab, 0x76,
            0xca, 0x82, 0xc9, 0x7d, 0xfa, 0x59, 0x47, 0xf0,
            0xad, 0xd4, 0xa2, 0xaf, 0x9c, 0xa4, 0x72, 0xc0,
            0xb7, 0xfd, 0x93, 0x26, 0x36, 0x3f, 0xf7, 0xcc,
            0x34, 0xa5, 0xe5, 0xf1, 0x71, 0xd8, 0x31, 0x15,
            0x04, 0xc7, 0x23, 0xc3, 0x18, 0x96, 0x05, 0x9a,
            0x07, 0x12, 0x80, 0xe2, 0xeb, 0x27, 0xb2, 0x75,
            0x09, 0x83, 0x2c, 0x1a, 0x1b, 0x6e, 0x5a, 0xa0,
            0x52, 0x3b, 0xd6, 0xb3, 0x29, 0xe3, 0x2f, 0x84,
            0x53, 0xd1, 0x00, 0xed, 0x20, 0xfc, 0xb1, 0x5b,
            0x6a, 0xcb, 0xbe, 0x39, 0x4a, 0x4c, 0x58, 0xcf,
            0xd0, 0xef, 0xaa, 0xfb, 0x43, 0x4d, 0x33, 0x85,
            0x45, 0xf9, 0x02, 0x7f, 0x50, 0x3c, 0x9f, 0xa8,
            0x51, 0xa3, 0x40, 0x8f, 0x92, 0x9d, 0x38, 0xf5,
            0xbc, 0xb6, 0xda, 0x21, 0x10, 0xff, 0xf3, 0xd2,
            0xcd, 0x0c, 0x13, 0xec, 0x5f, 0x97, 0x44, 0x17,
            0xc4, 0xa7, 0x7e, 0x3d, 0x64, 0x5d, 0x19, 0x73,
            0x60, 0x81, 0x4f, 0xdc, 0x22, 0x2a, 0x90, 0x88,
            0x46, 0xee, 0xb8, 0x14, 0xde, 0x5e, 0x0b, 0xdb,
            0xe0, 0x32, 0x3a, 0x0a, 0x49, 0x06, 0x24, 0x5c,
            0xc2, 0xd3, 0xac, 0x62, 0x91, 0x95, 0xe4, 0x79,
            0xe7, 0xc8, 0x37, 0x6d, 0x8d, 0xd5, 0x4e, 0xa9,
            0x6c, 0x56, 0xf4, 0xea, 0x65, 0x7a, 0xae, 0x08,
            0xba, 0x78, 0x25, 0x2e, 0x1c, 0xa6, 0xb4, 0xc6,
            0xe8, 0xdd, 0x74, 0x1f, 0x4b, 0xbd, 0x8b, 0x8a,
            0x70, 0x3e, 0xb5, 0x66, 0x48, 0x03, 0xf6, 0x0e,
            0x61, 0x35, 0x57, 0xb9, 0x86, 0xc1, 0x1d, 0x9e,
            0xe1, 0xf8, 0x98, 0x11, 0x69, 0xd9, 0x8e, 0x94,
            0x9b, 0x1e, 0x87, 0xe9, 0xce, 0x55, 0x28, 0xdf,
            0x8c, 0xa1, 0x89, 0x0d, 0xbf, 0xe6, 0x42, 0x68,
            0x41, 0x99, 0x2d, 0x0f, 0xb0, 0x54, 0xbb, 0x16
    };

    private static void subBytes(int[] state) {
        for (int i = 0; i < state.length; i++) {
            state[i] = SBOX[state[i]];
        }
    }
```

```
private static final int[] INV_SBOX = new int[256];
static {
    for (int i = 0; i < INV_SBOX.length; i++) {
        INV_SBOX[SBOX[i]] = i;
    }
}

private static void invSubBytes(int[] state) {
    for (int i = 0; i < state.length; i++) {
        state[i] = INV_SBOX[state[i]];
    }
}

private static void shiftRows(int[] state) {

    int[] temp = new int[4];

    for (int row = 0; row < 4; row++) {

        for (int col = 0; col < 4; col++) {
            int index = row + 4 * col;
            temp[col] = state[index];
        }

        for (int col = 0; col < 4; col++) {
            int newColumn = (col - row + 4) % 4;
            int index = row + 4 * newColumn;

            state[index] = temp[col];
        }
    }
}

private static void invShiftRows(int[] state) {

    int[] temp = new int[4];

    for (int row = 0; row < 4; row++) {

        for (int col = 0; col < 4; col++) {
            int index = row + 4 * col;
            temp[col] = state[index];
        }

        for (int col = 0; col < 4; col++) {
            int newColumn = (col + row + 4) % 4;
            int index = row + 4 * newColumn;

            state[index] = temp[col];
        }
    }
}

private static int xtime(int value)
{
    if((value &  0x80) != 0) {
        return ((value << 1) ^ 0x1B) & 0xFF;
    }
        else
        {
            return (value << 1) & 0xFF;
        }
}

private static int multiplyBy3(int value)
{
    return xtime(value) ^ value;
}
```

```
private static void mixColumns(int[] state) {

    int[] temp = new int[4];

    for (int col = 0; col < 4; col++) {

        for (int row = 0; row < 4; row++) {
            int index = col * 4 + row;
            temp[row] = state[index];
        }

        int result0 = xtime(temp[0])
                ^ multiplyBy3(temp[1])
                ^ temp[2]
                ^ temp[3];

        int result1 = temp[0]
                ^ xtime(temp[1])
                ^ multiplyBy3(temp[2])
                ^ temp[3];

        int result2 = temp[0]
                ^ temp[1]
                ^ xtime(temp[2])
                ^ multiplyBy3(temp[3]);

        int result3 = multiplyBy3(temp[0])
                ^ temp[1]
                ^ temp[2]
                ^ xtime(temp[3]);

        state[col * 4] = result0;
        state[col * 4 + 1] = result1;
        state[col * 4 + 2] = result2;
        state[col * 4 + 3] = result3;
    }
}

private static int multiplyBy9(int value) {
    int x2 = xtime(value);
    int x4 = xtime(x2);
    int x8 = xtime(x4);

    return x8 ^ value;
}

private static int multiplyBy11(int value) {
    int x2 = xtime(value);
    int x4 = xtime(x2);
    int x8 = xtime(x4);

    return x8 ^ x2 ^ value;
}

private static int multiplyBy13(int value) {
    int x2 = xtime(value);
    int x4 = xtime(x2);
    int x8 = xtime(x4);

    return x8 ^ x4 ^ value;
}

private static int multiplyBy14(int value) {
    int x2 = xtime(value);
    int x4 = xtime(x2);
    int x8 = xtime(x4);

    return x8 ^ x4 ^ x2;
}
```

```
private static void invMixColumns(int[] state) {
    int[] temp = new int[4];

    for (int col = 0; col < 4; col++) {
        for (int row = 0; row < 4; row++) {
            temp[row] = state[col * 4 + row];
        }

        int result0 = multiplyBy14(temp[0])
                ^ multiplyBy11(temp[1])
                ^ multiplyBy13(temp[2])
                ^ multiplyBy9(temp[3]);

        int result1 = multiplyBy9(temp[0])
                ^ multiplyBy14(temp[1])
                ^ multiplyBy11(temp[2])
                ^ multiplyBy13(temp[3]);

        int result2 = multiplyBy13(temp[0])
                ^ multiplyBy9(temp[1])
                ^ multiplyBy14(temp[2])
                ^ multiplyBy11(temp[3]);

        int result3 = multiplyBy11(temp[0])
                ^ multiplyBy13(temp[1])
                ^ multiplyBy9(temp[2])
                ^ multiplyBy14(temp[3]);

        state[col * 4] = result0;
        state[col * 4 + 1] = result1;
        state[col * 4 + 2] = result2;
        state[col * 4 + 3] = result3;
    }
}

private static void addRoundKey(int[] state, int[] roundKey)
{
    for (int i = 0; i < 16; i++) {
        state[i] = state[i] ^ roundKey[i];
    }
}

private  static void rotWord(int[] word)
{
    int temp = word[0];
    for (int i = 1; i < 4; i++) {
        word[i-1] = word[i];
    }
    word[3] = temp;
}

private  static void subWord(int[] word)
{
    for (int i = 0; i < word.length; i++) {
        word[i] = SBOX[word[i]];
    }
}

private static final int[] RCON = {
        0x01, 0x02, 0x04, 0x08, 0x10,
        0x20, 0x40, 0x80, 0x1B, 0x36
};

private static int[][] keyExpansion(int[] key) {
    int[][] words = new int[44][4];

    for (int i = 0; i < 4; i++) {
        for (int j = 0; j < 4; j++) {
```

```
            words[i][j] = key[i * 4 + j];
        }
    }

    for (int i = 4; i < 44; i++) {
        int[] temp = new int[4];
        for (int j = 0; j < 4; j++) {
            temp[j] = words[i-1][j];
        }
        if(i % 4 == 0)
        {
            rotWord(temp);
            subWord(temp);
            temp[0] = temp[0] ^ RCON[i / 4 - 1];
        }
        for (int j = 0; j < 4; j++) {
            words[i][j] = words[i - 4][j] ^ temp[j];
        }
    }
    return words;
}

private static int[] encrypt_block(int[] block, int[] key)
{
    int[][] keys;
    int[] state = new int[16];
    int[] roundKey = new int[16];
    if (block.length == 16 && key.length == 16)
    {
        keys = keyExpansion(key);
        for (int i = 0; i < 16; i++) {
            state[i] = block[i];
        }
        for (int i = 0; i < 4; i++) {
            for (int j = 0; j < 4; j++) {
                roundKey[i * 4 + j] = keys[i][j];
            }
        }
        addRoundKey(state, roundKey);
        for (int round = 1; round <= 9; round++)
        {
            subBytes(state);
            shiftRows(state);
            mixColumns(state);
            for (int i = 0; i < 4; i++) {
                for (int j = 0; j < 4; j++) {
                    roundKey[i * 4 + j] = keys[round * 4 + i][j];
                }
            }
            addRoundKey(state, roundKey);
        }
        subBytes(state);
        shiftRows(state);
        for (int i = 0; i < 4; i++) {
            for (int j = 0; j < 4; j++) {
                roundKey[i * 4 + j] = keys[10 * 4 + i][j];
            }
        }
        addRoundKey(state, roundKey);
    }
    else
        throw new IllegalArgumentException("Блок и ключ должны содержать 16 байт");
    return state;
}

private static int[] decrypt_block(int[] block, int[] key)
{
    if (block.length != 16 || key.length != 16) {
        throw new IllegalArgumentException(
```

```
                "Блок и ключ должны содержать 16 байт"
        );
    }

    int[][] keys = keyExpansion(key);
    int[] state = new int[16];
    int[] roundKey = new int[16];

    for (int i = 0; i < 16; i++) {
        state[i] = block[i];
    }

    for (int i = 0; i < 4; i++) {
        for (int j = 0; j < 4; j++) {
            roundKey[i * 4 + j] = keys[40 + i][j];
        }
    }

    addRoundKey(state, roundKey);

    for (int round = 9; round >= 1; round--) {

        invShiftRows(state);
        invSubBytes(state);

        for (int i = 0; i < 4; i++) {
            for (int j = 0; j < 4; j++) {
                roundKey[i * 4 + j] = keys[round * 4 + i][j];
            }
        }

        addRoundKey(state, roundKey);
        invMixColumns(state);
    }

    // Финальный раунд
    invShiftRows(state);
    invSubBytes(state);

    for (int i = 0; i < 4; i++) {
        for (int j = 0; j < 4; j++) {
            roundKey[i * 4 + j] = keys[i][j];
        }
    }

    addRoundKey(state, roundKey);

    return state;
}

private static int[] pad (int[] data)
{
    int padding = 16 - (data.length % 16);
    int[] newData = new int[data.length + padding];
    for (int i = 0; i < data.length; i++) {
        newData[i] = data[i];
    }
    for (int i = data.length; i < newData.length; i++) {
        newData[i] = padding;
    }
    return newData;
}

private static int[] unpad(int[] data)
{
    if(data.length == 0 || data.length % 16 != 0)
    {
        throw new IllegalArgumentException("Некорректная длина данных");
    }
```

```
    int padding = data[data.length - 1];
    if(padding < 1 || padding > 16 || padding > data.length)
    {
        throw new IllegalArgumentException("Некорректное дополнение");
    }
    for (int i = data.length - padding; i < data.length; i++) {
        if (data[i] != padding) {
            throw new IllegalArgumentException("Некорректное дополнение");
        }
    }
    int[] newData = new int[data.length - padding];

        for (int i = 0; i < newData.length; i++) {
            newData[i] = data[i];
        }

    return newData;
}

private static int[] encrypt_ecb(int[] data, int[] key)
{
    if (key.length != 16) {
        throw new IllegalArgumentException("Ключ должен содержать 16 байт");
    }

    int[] paddedData = pad(data);
    int[] encryptedData = new int[paddedData.length];

    for (int i = 0; i < paddedData.length; i += 16)
    {
        int[] block = new int[16];
        for (int j = 0; j < 16; j++) {
            block[j] = paddedData[i + j];
        }
        int[] encryptedBlock = encrypt_block(block, key);
        for (int j = 0; j < 16; j++) {
            encryptedData[i + j] = encryptedBlock[j];
        }
    }
    return encryptedData;
}

private static int[] decrypt_ecb(int[] data, int[] key)
{
    if (data.length == 0 || data.length % 16 != 0) {
        throw new IllegalArgumentException("Неправильная длина данных");
    }

    if (key.length != 16) {
        throw new IllegalArgumentException("Ключ должен содержать 16 байт");
    }

    int[] decryptedData = new int[data.length];

    for (int i = 0; i < data.length; i += 16)
    {
        int[] block = new int[16];

        for (int j = 0; j < 16; j++) {
            block[j] = data[i + j];
        }

        int[] decryptedBlock = decrypt_block(block, key);

        for (int j = 0; j < 16; j++) {
            decryptedData[i + j] = decryptedBlock[j];
        }
    }
```

```
    return unpad(decryptedData);
}

private static void printHex(int[] data)
{
    for (int value : data) {
        System.out.printf("%02X ", value);
    }
    System.out.println();
}

private static int[] encrypt_cfb(int[] data, int[] key, int[] iv)
{
    if (key.length != 16 || iv.length != 16) {
        throw new IllegalArgumentException("Ключ и IV должны содержать 16 байт");
    }

    int[] encryptedData = new int[data.length];
    int[] feedback = iv.clone();

    for (int i = 0; i < data.length; i += 16)
    {
        int[] encryptedFeedback = encrypt_block(feedback, key);

        int blockLength = Math.min(16, data.length - i);

        for (int j = 0; j < blockLength; j++) {
            encryptedData[i + j] = data[i + j] ^ encryptedFeedback[j];
        }

        if (blockLength == 16) {
            feedback = new int[16];
            for (int j = 0; j < 16; j++) {
                feedback[j] = encryptedData[i + j];
            }
        }
    }

    return encryptedData;
}

private static int[] decrypt_cfb(int[] data, int[] key, int[] iv)
{
    if (key.length != 16 || iv.length != 16) {
        throw new IllegalArgumentException("Ключ и IV должны содержать 16 байт");
    }

    int[] decryptedData = new int[data.length];
    int[] feedback = iv.clone();

    for (int i = 0; i < data.length; i += 16)
    {
        int[] encryptedFeedback = encrypt_block(feedback, key);

        int blockLength = Math.min(16, data.length - i);

        for (int j = 0; j < blockLength; j++) {
            decryptedData[i + j] = data[i + j] ^ encryptedFeedback[j];
        }

        if (blockLength == 16) {
            feedback = new int[16];
            for (int j = 0; j < 16; j++) {
                feedback[j] = data[i + j];
            }
        }
    }

    return decryptedData;
```

```
    }

    public static void main(String[] args)
    {
        int[] key = {
                0x00, 0x01, 0x02, 0x03,
                0x04, 0x05, 0x06, 0x07,
                0x08, 0x09, 0x0A, 0x0B,
                0x0C, 0x0D, 0x0E, 0x0F
        };

        int[] plaintext = new int[16];

        for (int i = 0; i < plaintext.length; i++) {
            plaintext[i] = i;
        }

        int[] encrypted = encrypt_ecb(plaintext, key);
        int[] decrypted = decrypt_ecb(encrypted, key);

        System.out.println("Исходные данные:");
        printHex(plaintext);

        System.out.println("Зашифрованные данные ECB:");
        printHex(encrypted);

        System.out.println("Расшифрованные данные ECB:");
        printHex(decrypted);

        System.out.println("Совпадают ли данные ecb: " +
                java.util.Arrays.equals(plaintext, decrypted));

        int[] iv = {
                0x00, 0x01, 0x02, 0x03,
                0x04, 0x05, 0x06, 0x07,
                0x08, 0x09, 0x0A, 0x0B,
                0x0C, 0x0D, 0x0E, 0x0F
        };

        int[] encryptedCFB = encrypt_cfb(plaintext, key, iv);
        int[] decryptedCFB = decrypt_cfb(encryptedCFB, key, iv);

        System.out.println("Зашифрованные данные CFB:");
        printHex(encryptedCFB);

        System.out.println("Расшифрованные данные CFB:");
        printHex(decryptedCFB);

        System.out.println("Совпадают ли данные cfb: " +
                java.util.Arrays.equals(plaintext, decryptedCFB));
    }

}
```

Вывод: в ходе лабораторной работы реализован симметричный блочный шифр AES-128 и режимы шифрования ECB и CFB (вариант 3). Реализованы алгоритмы шифрования, расшифрования и дополнения блоков по стандарту PKCS#7. Практически проверена корректность работы алгоритмов на данных различной длины.